



MECHATRONICS

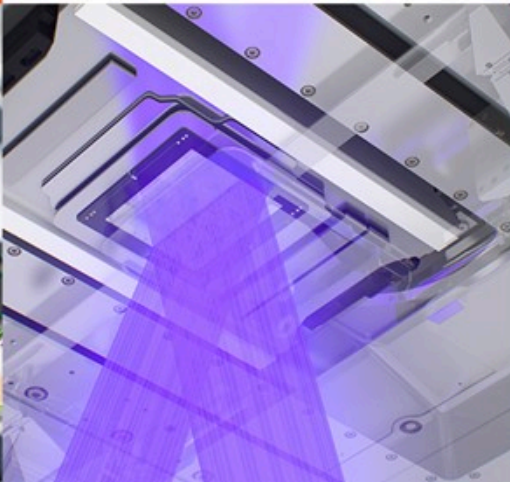


MATHWARE

ASSEMBLY



ELECTRONICS



SOFTWARE



Ranges



Ranges

- Why do we need ranges?
 - STL algorithms
 - Legacy Iterators vs C++20 iterators
- What is the definition of a range?
- Views and adaptors
- Sentinels
- Tombstones
- Compile time impact

C++23 mode is not enough

The compiler and its standard library must both implement each facility.

```
#include <version>

#if !defined(__cpp_lib_ranges_zip) || __cpp_lib_ranges_zip < 202110L
#error "This exercise requires a standard library with C++23 zip support"
#endif

#if !defined(__cpp_lib_ranges_enumerate) || \
    __cpp_lib_ranges_enumerate < 202302L
#error "This exercise requires C++23 enumerate support"
#endif
```

`-std=c++23` enables available C++23 features; it does not add missing library implementations. Use the course Docker image when the host library is older.

A trip down memory lane

Before introducing ranges, we will revisit the interfaces they evolved from:

1. Problems with legacy iterator pairs.
2. A few examples of legacy STL algorithms.

This gives us the context for the design choices made by C++20 ranges.

Problems with legacy iterators

Why was a new model needed?

Problem: Iterator Invalidation

There are no built-in safety checks or sentinels to prevent iterator use after invalidation

```
std::vector<int> v{1, 2, 3};  
auto it = v.begin();  
v.push_back(4);    // may reallocate, invalidates iterators  
int x = *it;       // undefined behaviour
```

Different containers have different invalidation rules.
A code change may invalidate an assumption

Container	push_back invalidates?
<code>std::vector</code>	Possibly
<code>std::list</code>	Never

Problem: Iterator mismatch

Legacy algorithms check iterator types, but cannot check whether both iterators belong to the same range.

```
std::vector<int> values{1, 2, 3};  
std::vector<int> other{4, 5, 6};  
  
int total = std::accumulate(  
    values.begin(),  
    other.end(), // Wrong container, but the iterator type matches  
    0);
```

Traversing from one container into another is undefined behaviour.

Problem: No sentinels

Iterators come in pairs of the same type: begin and end.

This makes some patterns awkward (e.g., searching until a terminator).

```
// Iterator pair requires finding an end iterator first.  
for (auto it = text; it != text + std::strlen(text); ++it) {  
    ...  
}
```

Upside: You could roll your own iterator

Downside: Complexity

Problem: Composability

Iterators do not compose well.

You typically end up writing loops instead of 'chaining' operations.

```
std::vector<int> result;  
for (auto it = v.begin(); it != v.end(); ++it) {  
    if (!predicate(*it)) continue;  
    auto transformed = transform(*it);  
    if (stop_condition(transformed)) break;  
    result.push_back(transformed);  
}
```

Upside: Memory usage

Downside: Readability

Granted: you could use 'intermediate containers' but you incur a memory and cpu loss.
Hence do not compose 'well'.

```
std::vector<int> intermediate;  
std::copy_if(v.begin(), v.end(), std::back_inserter(intermediate),  
            [](int x) { return x % 2 == 0; });  
  
std::vector<int> result;  
std::transform(intermediate.begin(), intermediate.end(), std::back_inserter(result),  
              [](int x) { return x * 10; });
```

Upside: Readability

Downside: CPU usage, Memory usage

Problem: Eager vs lazy

Standard algorithms execute immediately. When chained through intermediate containers, each stage must finish before the next begins.

Eager can be good:

- Predictable execution and side effects;
- The result is stored and can be reused;

Eager materialization can be bad:

- Intermediate containers cost memory and allocation;
- Later stages cannot stop earlier stages early;
- Unsuitable for unbounded streams or data larger than memory.

```
std::vector<int> intermediate;  
std::copy_if(v.begin(), v.end(), std::back_inserter(intermediate),  
            [](int x) { return x % 2 == 0; });  
  
std::vector<int> result;  
std::transform(intermediate.begin(), intermediate.end(), std::back_inserter(result),  
              [](int x) { return x * 10; });
```

What if only the first transformed match is needed?

- > `std::copy_if` already scanned the entire input
- > Stored every match before `std::transform` begins.

Problem: Handling failure

If `std::find` fails to find anything, you must manually compare with `end()`.

- Dereferencing the `end()` iterator is UB.
- Compiler does not help you here.

```
auto it = std::find(v.begin(), v.end(), 99);  
int x = *it;    // BUG if element not found
```

STL Algorithms use these problematic iterator pairs

Algorithms are decoupled from containers. The same algorithm works with any iterator type that provides the operations it requires.

```
std::vector<int> vec = {10, 20, 30, 40};  
auto it = std::find(vec.begin(), vec.end(), 30);  
  
std::list<int> lst = {10, 20, 30, 40};  
auto other = std::find(lst.begin(), lst.end(), 30);
```

`std::find` takes an input iterator pair and returns an iterator. Failure is represented by returning the supplied end iterator.

Algorithms that write need an output iterator

`std::copy_if` reads one iterator pair and writes matching elements through a third iterator.

```
std::vector<Order> completedOrders;

std::copy_if(orders.begin(), orders.end(),
             std::back_inserter(completedOrders),
             [](const Order& order) {
                 return order.status == "Completed";
             });
```

`std::back_inserter` adapts assignment into `completedOrders.push_back(...)`.
The algorithm does not know which container receives the output.

Algorithms that reduce produce one value

`std::accumulate` combines an iterator pair into a single value.

```
double total = std::accumulate(
    completedOrders.begin(), completedOrders.end(), 0.0,
    [](double sum, const Order& order) {
        return sum + order.totalPrice;
    });
```

- `0.0` is the initial value and determines the result type.
- The lambda combines the running total with the next element.

ex1.cpp

Historical context. Replace one raw loop with `std::copy_if` and `std::accumulate`.

The goal is to practise the legacy algorithm interface and expose the need for an intermediate container.

Legacy iterators: history

- **1994**: the STL was accepted into the draft C++ standard.
- **C++98**: iterator-based containers and algorithms were standardized.
- **C++20**: concepts introduced compiler-checkable iterator contracts.

The original iterator model separated traversal from storage:

```
std::find(first, last, value);
```

The algorithm does not need to know whether the iterators came from a `vector`, `list`, raw array, or user-defined type.

Today this pre-concepts model is called the **legacy iterator model**.

Legacy iterator named requirements

The standard described iterator contracts as **named requirements**:

cppreference	Current standard terminology
LegacyIterator	Cpp17Iterator
LegacyInputIterator	Cpp17InputIterator

A `LegacyIterator` is expected to be copyable and support:

```
*it;    // dereference
++it;    // advance
*it++;   // dereference, then advance
```

`std::iterator_traits<X>::difference_type` must be a signed integer type or `void`.

Legacy iterator categories: read and write

What an iterator allows an algorithm to do:

Category	Capability	Typical example
LegacyInputIterator	Read values in a single pass	<code>std::istream_iterator</code>
LegacyOutputIterator	Write values in a single pass	<code>std::back_insert_iterator</code>

Input and output iterators are the weakest useful categories: one-way stream

```
std::copy(input_begin, input_end, output_begin);
```

`std::copy` reads through an input iterator and writes through an output iterator.

Legacy iterator categories: traversal

Each category adds guarantees and operations to the one above it:

Category	Additional capability	Typical example
<code>LegacyForwardIterator</code>	Multi-pass forward traversal	<code>std::forward_list</code>
<code>LegacyBidirectionalIterator</code>	Move backward with <code>--it</code>	<code>std::list</code>
<code>LegacyRandomAccessIterator</code>	Constant-time jumps and distance	<code>std::vector</code>
<code>LegacyContiguousIterator</code>	Elements are adjacent in memory	pointers, <code>std::vector</code>

Algorithms require the weakest category for their work. Possible overloads for stronger categories.

Legacy algorithms

Lets take a look at 1 declaration of `std::find`

```
template<class InputIterator, class T = iterator_traits<InputIterator>::value_type>  
constexpr InputIterator find(InputIterator first, InputIterator last, const T& value);
```

If we then look up `InputIterator`:

If an algorithm's template parameter is named `InputIterator`,
the template argument shall meet the Cpp17InputIterator requirements ([input.iterators]).

This contract is NOT enforced by the compiler. [cppreference](#):

InputIt must meet the requirements of LegacyInputIterator.

5 Core traits

To be useable by legacy algorithms (e.g. `std::find`), types NEED to have:

- `value_type`: The type of the element the iterator points to.
- `difference_type`: A signed integer type that can represent the distance between two iterators.
- `reference`: The reference type of the element (usually `value_type&`).
- `pointer`: The pointer type of the element (usually `value_type*`).
- `iterator_category`: A tag (e.g., `std::forward_iterator_tag`) that tells algorithms which operations the iterator supports, enabling compile-time optimizations (tag dispatching).

Deprecated since C++17:

```
class Iterato
```

Base class that provides the 5 core traits.

There are layers of legacy

```
template <typename Iterator>
struct iterator_traits {
    using difference_type    = typename Iterator::difference_type;
    using value_type         = typename Iterator::value_type;
    using pointer            = typename Iterator::pointer;
    using reference          = typename Iterator::reference;
    using iterator_category = typename Iterator::iterator_category;
};

template <typename T>
struct iterator_traits<T*> {
    using difference_type    = std::ptrdiff_t;
    using value_type         = std::remove_cv_t<T>;
    using pointer            = T*;
    using reference          = T&;
    using iterator_category = std::random_access_iterator_tag;
};
```

```
#include <iterator>
struct InvalidIterator {
    // Missing: using difference_type = std::ptrdiff_t;
    using value_type = int;           // Present
    using pointer = int*;            // Present
    using reference = int&;          // Present
    using iterator_category = std::forward_iterator_tag; // Present
}
int main() {
    static_assert(std::is_same_v<std::iterator_traits<InvalidIterator>::value_type, int>);
}
```

```
<source>:10:73: error: 'value_type' is not a member of 'std::iterator_traits<InvalidIterator>'
  10 |         static_assert(std::is_same_v<std::iterator_traits<InvalidIterator>::value_type, int>);
      |
```

Tag dispatching

```
template <typename Iterator, typename Distance>
void my_advance(Iterator& it, Distance n) {
    using category = typename std::iterator_traits<Iterator>::iterator_category;
    detail::advance_impl(it, n, category{});
}
```

```

namespace detail {
    // Overload for Input and Forward Iterators: O(N) complexity
    template <typename Iterator, typename Distance>
    void advance_impl(Iterator& it, Distance n, std::forward_iterator_tag) {
        // Must step through one by one
        while (n > 0) {
            ++it;
            --n;
        }
    }
    // Overload for Bidirectional Iterators: O(N) complexity but supports negative n
    template <typename Iterator, typename Distance>
    void advance_impl(Iterator& it, Distance n, std::bidirectional_iterator_tag) {
        if (n > 0) {
            while (n-- > 0) ++it;
        } else {
            while (n++ < 0) --it;
        }
    }
    template <typename Iterator, typename Distance>
    void advance_impl(Iterator& it, Distance n, std::random_access_iterator_tag) {
        // Direct jump using pointer arithmetic
        it += n;
    }
}

```

```

struct MyIntIterator {
    using iterator_category = std::forward_iterator_tag;
    using difference_type   = std::ptrdiff_t;
    using value_type        = int;
    using pointer            = int*;
    using reference          = int&;
    explicit MyIntIterator(pointer ptr) : m_ptr(ptr) {}
    reference operator*() const { return *m_ptr; }
    pointer operator->() { return m_ptr; }
    MyIntIterator& operator++() {
        m_ptr++;
        return *this;
    }
    MyIntIterator operator++(int) {
        MyIntIterator tmp = *this;
        ++(*this);
        return tmp;
    }
    friend bool operator==(const MyIntIterator& a, const MyIntIterator& b) {
        return a.m_ptr == b.m_ptr;
    }
    friend bool operator!=(const MyIntIterator& a, const MyIntIterator& b) {
        return a.m_ptr != b.m_ptr;
    }
private:
    pointer m_ptr;
};

```

```
#include <iostream>
#include <vector>
#include <string>
#include <iterator>
struct File { //Invisible to the use
    std::string name;
    std::string content;
};
struct Directory {
private:
    std::vector<File> files;
public:
    TextIterator begin() const { return TextIterator(files.begin()); }
    TextIterator end() const { return TextIterator(files.end()); }
};
```

We want to loop over a set of files in a directory. The concept of a 'File' is an implementation detail the user of our lib does not know of this.

```

class TextIterator {
private:
    std::vector<File>::const_iterator file_it;
public:
    using iterator_category = std::forward_iterator_tag;
    using value_type        = std::string;
    using difference_type    = std::ptrdiff_t;
    using pointer            = const std::string*;
    using reference          = const std::string&;
    explicit TextIterator(std::vector<File>::const_iterator it) : file_it(it) {}
    // intercept the dereference to return the string, not the File
    reference operator*() const { return file_it->content; }
    pointer operator->() const { return &(file_it->content); }
    TextIterator& operator++() {
        ++file_it;
        return *this;
    }
    TextIterator operator++(int) {
        TextIterator tmp = *this;
        ++file_it;
        return tmp;
    }
    friend bool operator==(const TextIterator& a, const TextIterator& b) {
        return a.file_it == b.file_it;
    }
    friend bool operator!=(const TextIterator& a, const TextIterator& b) {
        return a.file_it != b.file_it;
    }
};

```

```
int main() {  
    Directory my_dir = {  
        {  
            {"file1.txt", "Hello from the first file."},  
            {"file2.txt", "This is the second file."},  
            {"file3.txt", "And here is the third!"}  
        }  
    };  
    // The loop only sees std::strings. The File structs are invisible.  
    for (const std::string& text : my_dir) {  
        std::cout << text << "\n";  
    }  
    return 0;  
}
```

Exercise: ex2.cpp

Complete Channellterator so an STL algorithm can traverse one channel of interleaved RGB image data.

- Define the five legacy iterator traits.
- Implement dereference and increment for the channel memory layout.
- Use `std::accumulate` to sum the green channel.

The goal is to practise the legacy iterator named requirements and hide a non-trivial storage layout behind a familiar iterator interface.

At this point you should be thinking: Non compiler enforced contract?

```
using iterator_category = std::forward_iterator_tag;  
using value_type        = std::string;  
using difference_type    = std::ptrdiff_t;  
using pointer           = const std::string*;  
using reference          = const std::string&;
```

Feels like something exceptionally bad.

Here ends the history lesson

And that is how things were in the past. Lets not dwell on the past.

Onwards to concept-checked iterators, ranges, views, adaptors, and range algorithms.

What caused the legacy mass extinction event?

- **Concepts:** express and check iterator and range requirements.
- **Customization-point objects:** provide one constrained interface.
- **Sentinel concept:** the end condition can have a different type from the iterator.
- **Range objects:** package a traversal source instead of passing two independent iterators.
- **Views:** store lazy transformations as lightweight range objects.
- **Adaptor closures and `operator|`:** compose views into readable pipelines.
- **Perfect forwarding and CTAD:** infer the complex wrapper types.

C++20 Iterators

C++20: Enter concepts and type constraining.

- differentiate the weak, 'legacy' unchecked named-requirements from the new, compiler-enforced `std::` concepts.

Lets take a look at those concepts

Input / Output Iterators

The most basic iterators. Support single-pass read (Input) or write (Output) operations. Reading or writing consumes the element (e.g., stream iterators)..

```
template< class I >
concept input_iterator =
    std::input_or_output_iterator<I> && /* *it and weakly incrementable*/
    std::indirectly_readable<I> && /* *it, repeated *it*/
    std::derived_from<ITER_CONCEPT<I>, std::input_iterator_tag>;
```

Forward iterators

Multi-pass forward iteration. Safe to copy the iterator and iterate over the same range multiple times without consuming it.

```
template< class I >
    concept forward_iterator =
        std::input_iterator<I> &&
        std::derived_from</*ITER_CONCEPT*/<I>, std::forward_iterator_tag> &&
        std::incrementable<I> /*equality preserving ++ operator*/ &&
        std::sentinel_for<I, I>;
```

Why **incrementable** enables multiple passes

input_iterator only requires weakly_incrementable: copies may share state.

incrementable strengthens this contract:

```
template<class I>
concept incrementable =
    std::regular<I> && // copyable and equality-comparable
    std::weakly_incrementable<I> &&
    requires(I i) { { i++ } -> std::same_as<I>; }; // post increment
```

Increment is **equality-preserving**: if `a == b`, then `++a == ++b`.

Advancing one copy must not change what another copy refers to. This semantic multi-pass promise cannot be proved by the compiler.

Bidirectional Iterators

Forward guarantees, plus the ability to iterate backwards step-by-step (supports --it).

```
template< class I >
    concept bidirectional_iterator =
        std::forward_iterator<I> &&
        std::derived_from</*ITER_CONCEPT*/<I>, std::bidirectional_iterator_tag> &&
        requires(I i) {
            { --i } -> std::same_as<I&>;
            { i-- } -> std::same_as<I>;
        };
};
```

Random access iterators

Constant time $O(1)$ jumps ($it + n$), distance calculations, and relational comparisons ($it1 < it2$). Time complexity is a promise by the author.

```
template< class I >
    concept random_access_iterator =
        std::bidirectional_iterator<I> &&
        std::derived_from</*ITER_CONCEPT*/<I>, std::random_access_iterator_tag> &&
        std::totally_ordered<I> &&
        std::sized_sentinel_for<I, I> &&
        requires(I i, const I j, const std::iter_difference_t<I> n) {
            { i += n } -> std::same_as<I&>;
            { j +  n } -> std::same_as<I>;
            { n +  j } -> std::same_as<I>;
            { i -= n } -> std::same_as<I&>;
            { j -  n } -> std::same_as<I>;
            {  j[n]  } -> std::same_as<std::iter_reference_t<I>>;
        };
```

Contiguous Iterators

Random access and guarantees elements are adjacent in physical memory (e.g., `std::vector`, `std::array`).

```
template< class I >
    concept contiguous_iterator =
        std::random_access_iterator<I> &&
        std::derived_from</*ITER_CONCEPT*/<I>, std::contiguous_iterator_tag> &&
        std::is_lvalue_reference_v<std::iter_reference_t<I>> && /* detect proxy*/
        std::same_as<std::iter_value_t<I>,
                    std::remove_cvref_t<std::iter_reference_t<I>>> &&
        requires(const I& i) {
            { std::to_address(i) } ->
            std::same_as<std::add_pointer_t<std::iter_reference_t<I>>>;
        };
};
```

- C-API Interoperability; `memcpy`

std::to_address

std::to_address(i) exposes the raw pointer to the element referenced by i.
A pointer-backed iterator usually enables it through operator->():

```
T& operator*() const { return *ptr; }  
T* operator->() const { return ptr; }
```

The concept checks that dereference returns a real T&, not a proxy, and that std::to_address(i) returns the corresponding T*.

The iterator author promises:

```
std::to_address(i)      == std::addressof(*i)  
std::to_address(i + n) == std::to_address(i) + n
```

The address relationships are semantic requirements that the compiler cannot prove.

Iterator concepts: two branches

`input_or_output_iterator` provides the common cursor operations: dereference and (weak) increment. Reading and writing are separate capabilities.

```
input_or_output_iterato
|-- output_iterator<I, T>    writable, usually single-pass
`-- input_iterator          readable, possibly single-pass
    |-- forward_iterator    readable and multi-pass
```

- The standard traversal hierarchy describes **readable** (input) iterators.
- Output capability is orthogonal and depends on the value type `T`.
- There is no standard multi-pass output concept; you could combine `output_iterator` with `incrementable` and `sentinel_for`.

By explicitly specifying using `iterator_concept = std::forward_iterator_tag`, the author of `forward_list` is telling the compiler:

Even though my syntax looks like a basic incrementable type,
I guarantee that copying it is safe and it satisfies multi-pass semantics.

The tag is needed

Input Iterator (`std::input_iterator_tag`): Supports `++`, but is single-pass.

Forward Iterator (`std::forward_iterator_tag`): Supports `++`, and is multi-pass.

Using C++20 Iterators

Constrain type I1 and I2 to random_access_iterator.
Similar to the tag dispatch example earlier.

```
template <std::random_access_iterator I1,  
          std::random_access_iterator I2>  
bool same_distance(I1 first1, I1 last1,  
                  I2 first2, I2 last2) {  
    // O(1) pointer arithmetic guaranteed  
    return (last1 - first1) == (last2 - first2);  
}
```

This contract is checked by the compiler!

Associated types of a modern iterator

C++20 concepts inspect expressions and associated types. A readable iterator normally declares the types that cannot be inferred reliably:

```
struct iterator {  
    using value_type = Record;           // value produced by reading  
    using difference_type = std::ptrdiff_t; // signed distance type  
    using iterator_concept = std::forward_iterator_tag;  
  
    Record const& operator*() const;  
    iterator& operator++();  
};
```

- `reference` is usually derived from `decltype(*it)`.
- `pointer` is not required by the C++20 iterator concepts.
- `iterator_category` to support legacy algorithms and their tag dispatch.

Why `difference_type` is required

Iterator subtraction and counting need a signed type: `end - begin` is positive, while `begin - end` is negative.

```
using difference_type = std::ptrdiff_t;  
  
auto distance = last - first; // may span the complete addressable range  
std::ranges::advance(it, -3); // negative for bidirectional iterators
```

A custom iterator may instead use another signed integer type when its domain is smaller or larger than pointer distance. `std::iter_difference_t<I>` exposes the selected type to algorithms.

iterator_concept and iterator_category

The two tags serve different consumers:

```
using iterator_concept  = std::forward_iterator_tag; // C++20 concepts
using iterator_category = std::forward_iterator_tag; // legacy algorithms
```

- `iterator_concept` states the strongest C++20 iterator concept modeled.
- `iterator_category` enables pre-C++20 tag dispatch.
- If `iterator_concept` is absent, the concepts machinery falls back to `iterator_category`, then applies additional fallback rules for pointers.

The tags declare capability; semantic promises remain the author's responsibility.

When worlds collide:

```
struct ModernIter {  
    // We only provide the strict minimum aliases required for C++20 Concepts  
    using value_type = int;  
    using difference_type = std::ptrdiff_t;  
    using iterator_concept = std::input_iterator_tag;  
    // INTENTIONALLY MISSING: // using pointer = int*; // using reference = int&; // using iterator_category = std::input_iterator_tag;  
    int* ptr;  
    int& operator*() const { return *ptr; }  
    ModernIter& operator++() { ++ptr; return *this; }  
    // C++20 allows post-increment to return void for input iterators.  
    void operator++(int) { ++ptr; }  
    bool operator==(const ModernIter& o) const = default;  
};  
static_assert(std::input_iterator<ModernIter>);\br/>void test() {  
    int arr[] = {1, 2, 3};  
    auto it2 = std::find(ModernIter{arr}, ModernIter{arr+3}, 2);  
}
```

```
stl_algo.h:3863:13: error: no type named 'value_type' in 'struct std::iterator_traits<ModernIter>'  
3863 |         using _ValT = typename iterator_traits<_InputIterator>::value_type;
```

Exercise: ex3.cpp

Complete TlvIterator so it traverses variable-length TLV packets in a raw byte buffer.

- Define the C++20 aliases, including `iterator_concept`.
- Verify the iterator with `std::forward_iterator`.
- Create a `std::ranges::subrange` and calculate the packet count, total payload size, and type checksum.

The goal is to practise building a concept-checked C++20 iterator and exposing structured packets from a variable-length binary layout as a range.

Ranges

The ranges library has three cooperating parts:

1. **Range concepts** describe what a source can do.
2. **Views and adaptors** lazily describe a traversal.
3. **Algorithms** consume a range and perform work immediately.

```
container / stream
    | views::filter | views::transform
    v
    lazy view
    | ranges::find / ranges::copy / ranges::sort
    v
iterator, result object, mutation, or materialized output
```

A **range** is

1. An **iterator** and a **sentinel** that designate the beginning and end of the computation
2. An **iterator** and a **count** that designate the beginning and the number of elements to which the computation is to be applied

A **range** is any object that can produce a begin iterator and an end sentinel. Their types may differ.

```
template< class T >
concept range = requires( T& t ) {
    ranges::begin(t);
    ranges::end (t);
};
```

Range concepts have two dimensions

Traversal strength comes from the range's **iterator**:

```
input_range -> forward_range -> bidirectional_range  
            -> random_access_range -> contiguous_range
```

Other concepts describe the **range as a whole** and are orthogonal:

- `sized_range` : `ranges::size(r)` is available.
- `common_range` : iterator and sentinel have the same type.
- `borrowed_range` : iterators may outlive the range object.
- `view` : a range designed for cheap construction and movement.

A range can be forward but not sized, or contiguous but not borrowed.

Sized ranges and sized sentinels

`sized_range<R>` means `std::ranges::size(r)` is valid and constant time.

```
template<std::ranges::sized_range R>
void reserve_for(R&& range, std::vector<int>& output) {
    output.reserve(std::ranges::size(range));
}
```

A range may know its size without using a sized sentinel, for example through a container's `size()` member.

Common ranges

A `common_range` uses the same type for `begin()` and `end()` :

```
template<class R>
concept common_range = std::ranges::range<R> &&
    std::same_as<std::ranges::iterator_t<R>,
        std::ranges::sentinel_t<R>>;
```

This matters when an API needs an iterator pair of one type, constructs a `subrange<I, I>`, or walks backward from `end()`. A sentinel-based range may still be forward, sized, or even random access without being common.

Customization points

A customization point gives generic code one stable interface while allowing user-defined types to provide type-specific behavior.

```
auto first = std::ranges::begin(range);  
auto count = std::ranges::size(range);
```

The caller always uses the qualified `std::ranges` interface.

The customization logic remains in the namespace and does NOT leak into the caller's overload set.

T E; ranges::begin(E)

- If T is an array type, ranges::begin(E) is expression-equivalent to t + 0.
- If auto(t.begin()) is a valid expression whose type models input_or_output_iterator, ranges::begin(E) is expression-equivalent to auto(t.begin()).
- If T is a class or enumeration type and auto(begin(t)) is a valid expression whose type models input_or_output_iterato

```

// namespace infiltration
#include <iostream>
#include <vector>
#include <iterator>
namespace vendor { // --- Third-Party Graphics Library ---
    struct Color { float r, g, b; };
    template <typename Target>
    void begin(Target& t) {
        std::cout << "vendor::begin -> Started rendering batch!\n";
    }
}
template <typename Container>
void process_elements(Container& c) {
    using std::begin; // "Two-Step" idiom (unrelated)
    auto it = begin(c);
    std::cout << "Processing elements...\n";
}
int main() {
    std::vector<vendor::Color> colors = {{1,0,0}, {0,1,0}};
    process_elements(colors);
}

```

```
<source>:14:20: error: call of overloaded 'begin(std::vector<vendor::Color>&)' is ambiguous  
14 |         auto it = begin(c);
```

Niebloid: customization point

Eric Niebler

[Ranges-v3](#). The reference implementation for c++ ranges
cppreference also uses this term 'customization point'

ADL applies to an **unqualified function call** such as `begin(value)`.
It does not apply when calling an already-selected function object:

```
std::ranges::begin(value); // calls begin.operator()(value)
```

```

struct _Begin
{
private:
    template<typename _Tp>
        static constexpr bool
        _S_noexcept()
        {
            ...
        }
public:
    template<__maybe_borrowed_range _Tp>
        requires is_array_v<remove_reference_t<_Tp>>> || __member_begin<_Tp>
        || __adl_begin<_Tp>
        [[nodiscard, __gnu__::__always_inline__]]
        constexpr auto
        operator()( _Tp&& __t) const noexcept(_S_noexcept<_Tp>())
        {
            if constexpr (is_array_v<remove_reference_t<_Tp>>>)
            {
                static_assert(is_lvalue_reference_v<_Tp>);
                return __t + 0;
            }
            else if constexpr (__member_begin<_Tp>)
                return __t.begin();
            else
                return begin(__t);
        }
};

```

What is a range?

A range is any object that exposes an iteration domain.

The important idea is that we describe the whole traversal with one object.

```
std::vector<int> v = {5, 1, 4, 2};  
std::ranges::sort(v);  
std::ranges::for_each(v, [](int x) { std::cout << x << ' '; });
```

The old style was:

```
std::sort(v.begin(), v.end());
```

The new style says:

```
std::ranges::sort(v);
```

What can a range express?

A range can represent a container, a subrange, a view, or a custom stream of values.

```
std::vector<int> v = {1, 2, 3, 4, 5, 6};  
auto r = std::ranges::subrange(v.begin(), v.begin() + 4);  
for (int x : r) {  
    std::cout << x << ' '  
}  
}
```

We are no longer forced to think in terms of a pair of iterators that must match exactly.

Sentinels

If a range ends on a special terminator, a sentinel can describe that end.

```
struct semicolon_sentinel {};  
  
bool operator==(const char* it, semicolon_sentinel)  
{  
    return *it == ';';  
}  
  
const char* text = "abc;def";  
auto r = std::ranges::subrange(  
    text,  
    semicolon_sentinel{}  
);
```

The iterator can stay lightweight while the end condition is expressed by a sentinel.

A sentinel can represent parser exhaustion

```
struct element_iterator {  
    explicit element_iterator(const char* p) : cur_(p) {  
        current_ = read_next();  
    }  
    const Element& operator*() const { return *current_; }  
    element_iterator& operator++() {  
        current_ = read_next(); return *this; }  
    friend bool operator==(const element_iterator& it,  
                           std::default_sentinel_t) {  
        return !it.current_.has_value(); }  
    const char* cur_ = nullptr;  
    std::optional<Element> current_;  
    std::optional<Element> read_next(); };
```

`default_sentinel` carries no state; comparison asks the iterator whether the source has been exhausted.

subrange turns iterator boundaries into a range

`std::ranges::subrange` is a view that stores an iterator and sentinel. It does not transform elements or own them.

```
auto tail = std::ranges::subrange(found, values.end());
std::ranges::sort(tail);

auto first_four = std::ranges::subrange(
    values.begin(), values.begin() + 4);
```

Class template argument deduction normally supplies the iterator and sentinel types. Their types may differ; matching types are required only when the result must be a `common_range`.

Sized and unsized subrange

The third template argument records whether the size is available:

```
using std::ranges::subrange;  
using std::ranges::subrange_kind;  
  
subrange first{begin, end};           // size inferred when possible  
subrange counted{begin, end, count};  // explicitly stores a size  
  
static_assert(std::ranges::sized_range<decltype(counted)>);
```

`subrange_kind::sized` requires a sized sentinel or an explicitly supplied count. The iterators still refer to external storage, so their invalidation and lifetime rules remain unchanged.

Views

A view is a range designed for cheap construction, move, and composition. It may refer to external data or own its source.

```
std::vector<int> v = {1, 2, 3, 4, 5, 6};  
auto r1 = std::ranges::subrange(v);  
auto r2 = std::ranges::take_view(r1, 4);  
auto r3 = std::ranges::filter_view(r2,  
    [](int x) { return x % 2 == 0; });  
  
for (int x : r3) {  
    std::cout << x << ' '  
}
```

The important point is not the type. The important point is that the range remains a view.

The `std::ranges::view` concept

The concept checks that a type is a range, is movable, and has opted into the view model:

```
template<class T>
concept view =
    std::ranges::range<T> &&
    std::movable<T> &&
    std::ranges::enable_view<T>;
```

`enable_view<T>` distinguishes lightweight view types from ordinary ranges. The concept does not itself mean "lazy", "non-owning", or "read-only".

subrange is a view

```
std::vector<int> values{1, 2, 3, 4};

auto slice = std::ranges::subrange(
    values.begin() + 1, values.end());

static_assert(std::ranges::range<decltype(slice)>);
static_assert(std::ranges::view<decltype(slice)>);

static_assert(std::ranges::range<decltype(values)>);
static_assert(!std::ranges::view<decltype(values)>);
```

Both types are ranges. `subrange` is also a view because it is a lightweight, movable range that stores only its iterator, sentinel, and possibly a size.

How a type becomes a view

Standard view types, including `subrange`, already opt in. A custom type usually inherits from `std::ranges::view_interface`:

```
class records_view
    : public std::ranges::view_interface<records_view> {
public:
    iterator begin();
    sentinel end();
};

static_assert(std::ranges::view<records_view>);
```

`view_interface` supplies the view marker and may provide convenience members such as `empty()`, `front()`, `back()`, and `operator[]` when supported by the range's capabilities.

Wrapping the parser iterator in a view

```
class element_range
    : public std::ranges::view_interface<element_range> {
public:
    explicit element_range(const char* text) : text_(text) {}

    element_iterator begin() const {
        return element_iterator{text_};
    }
    std::default_sentinel_t end() const { return {}; }

private:
    const char* text_;
};

static_assert(std::ranges::view<element_range>);
```

The range supplies the iterator; the stateless sentinel represents exhaustion.

Views

We now know:

- A view is a lightweight range object.
- It *may* own the data it is looking at.
- It simply gives a new way to observe a source range.

```
std::vector<int> data{1, 2, 3, 4, 5, 6};  
  
auto first_three = std::ranges::subrange(data.begin(), data.begin() + 3);  
  
for (int x : first_three) {  
    std::cout << x << ' ';    // 1 2 3  
}
```

The vector still owns the elements.

Composing views

One key feature of views is composability:

- range → transform → filter → slice → consume
- Do not copy the underlying data
- The view is copied, not the container

```
std::vector<int> v{1, 2, 3, 4, 5, 6};  
auto r1 = std::ranges::subrange(v);  
auto r2 = std::ranges::take_view(r1, 4);  
auto r3 = std::ranges::filter_view(r2, [](int x){ return x % 2 == 0; });  
  
for (int x : r3)  
    std::cout << x << ' ';
```

Every step wraps the previous range without materializing a new container.

Pipeline syntax

Use the pipe operator to build a range expression from left to right.

```
std::vector<int> v{1, 2, 3, 4, 5, 6};  
auto r = v  
    | std::ranges::views::take(4)  
    | std::ranges::views::filter([](int x) { return x % 2 == 0; });  
for (int x : r)  
    std::cout << x << ' ';
```

Laziness has observable consequences

Work happens during iteration, and most views continue to observe their source.

```
std::vector<int> data{1, 2, 3, 4};
auto evens = data | std::views::filter([](int value) {
    std::cout << "testing " << value << '\n';
    return value % 2 == 0;
});

data[1] = 20;           // the view refers to data
for (int value : evens) { // predicate runs here
    std::cout << value << '\n';
}
```

Do not structurally modify the source while iterating if that invalidates the iterators used by the view. Ranges do not remove container invalidation rules.

A pipeline keeps only supported guarantees

Views may weaken the capabilities of their input range.

```
auto transformed = data | std::views::transform(square);  
static_assert(std::ranges::random_access_range<decltype(transformed)>);  
  
auto filtered = data | std::views::filter(is_even);  
static_assert(std::ranges::bidirectional_range<decltype(filtered)>);  
static_assert(!std::ranges::random_access_range<decltype(filtered)>);
```

Filtering cannot provide constant-time indexing: finding the next match requires searching. Constrain consumers by the guarantees they actually need.

Adaptors

An adaptor is a range factory.

It takes one range and returns another range-like object, usually a lightweight view.

The purpose is to keep the pipeline lazy and composable:

range → filter → consume

```
std::vector<int> data{1, 2, 3, 4, 5, 6};

auto evens = data
    | std::views::filter([](int x) { return x % 2 == 0; });

for (int x : evens) {
    std::cout << x << ' ';    // 2 4 6
}
```

`std::views::filter` is actually an adapter not the view

What an adaptor really does

An adaptor answers the question:

"Given a range, can I produce a new view with a different shape?"

Common examples:

- `std::views::filter`: keep elements satisfying a predicate
- `std::views::transform`: map each element to another value
- `std::views::take`: stop after N elements

Type-wise, the adaptor and the view are different objects:

```
auto is_even = [](int value) { return value % 2 == 0; };  
auto adaptor = std::views::filter(is_even); // unspecified closure type  
auto filtered = data | adaptor;             // concrete view type
```

`std::views::filter` is a factory object. Its result wraps the source and predicate in a `std::ranges::filter_view`; it does not copy matching elements.

```
std::vector<int> data{1, 2, 3, 4, 5, 6};

auto result = data
    | std::views::filter([](int x) { return x % 2 == 0; })
    | std::views::transform([](int x) { return x * 10; })
    | std::views::take(2);

for (int x : result) {
    std::cout << x << ' ';    // 20 40
}
```

The important idea is that every step is a view, and each view remains cheap to store and compose.

The concrete view type

```
int main() {  
    std::vector data{1, 2, 3, 4, 5, 6};  
    auto evens = data | std::views::filter([](int x) { return x % 2 == 0; });  
    std::cout << get_type_name<decltype(evens)>() << "\n\n";  
    return 0;  
}
```

```
std::ranges::filter_view<std::ranges::ref_view<std::vector<int, std::allocator<int> > >, main::{lambda(int)#1}>
```

The concrete type records both the source representation (`ref_view`) and the predicate type. Stateless predicates can usually benefit from empty base optimization (EBO).

This spelling is practical because of Class Template Argument Deduction (CTAD, C++17). In interfaces, avoid naming the complete type: use `auto` return type o constrain the result as a range.

```
template< ranges::input_range V,  
         std::indirect_unary_predicate<ranges::iterator_t<V>> Pred >  
    requires ranges::view<V> && std::is_object_v<Pred>  
class filter_view  
    : public ranges::view_interface<filter_view<V, Pred>>
```

std::ranges::ref_view

The adaptor wraps the range in a `ref_view` to avoid copying the vector.

The view stores a reference to the original object, not a duplicate of the container.

```
std::vector<int> v = {1, 2, 3, 4};
auto is_even = [](int x){ return x % 2 == 0; };
auto factory = std::views::filter(is_even);
auto view = v | factory;

static_assert(
    std::same_as<
        decltype(view),
        std::ranges::filter_view<std::ranges::ref_view<decltype(v)>, decltype(is_even)>
    >
);
```

```
std::ranges::filter_view<std::ranges::ref_view<vector<int>>, predicate>
```

The predicate is the view transformation.

The storage strategy is `ref_view`, so the vector is not copied.

std::ranges::views::all

`std::views::all` is a range adaptor that decides how to represent the source.

- If the input is already a view: copy or move that view.
- If the input is a non-view lvalue: `std::ranges::ref_view<T>`.
- If the input is a non-view rvalue: `std::ranges::owning_view<T>`.

```
int main() {
    std::vector<int> v = {1, 2, 3};
    auto view1 = std::views::all(v); // l-value case: keep a reference to the existing vector
    static_assert(std::is_same_v<decltype(view1), std::ranges::ref_view<std::vector<int>>>,
        "Expected ref_view for lvalue"
    );
    auto view2 = std::views::all(std::vector<int>{1, 2, 3}); // r-value case: move the vector into an owning view
    static_assert(
        std::is_same_v<decltype(view2), std::ranges::owning_view<std::vector<int>>>,
        "Expected owning_view for rvalue"
    );
    return 0;
}
```

Common view adaptors

Adaptors accept a range now or return a closure for pipe syntax.

```
views::filter(range, predicate)    // keep matching elements
views::transform(range, function)  // map each element
views::take(range, count)          // first count elements
views::drop(range, count)          // skip count elements
views::take_while(range, predicate)
views::drop_while(range, predicate)
views::reverse(range)
```

Their curried forms are equivalent:

```
auto active = views::filter(is_active);
auto result = records | active | views::transform(to_name);
```

Algorithms

Range algorithms execute immediately. Most accept the whole range and optional callables or projections:

```
ranges::find(range, value, projection)          -> borrowed_iterator_t<R>
ranges::find_if(range, predicate, projection)-> borrowed_iterator_t<R>
ranges::for_each(range, function, projection)-> in_fun_result<I, F>
ranges::copy(range, output_iterator)           -> in_out_result<I, O>
ranges::sort(range, comparator, projection)    -> borrowed_iterator_t<R>
```

Use an algorithm for a terminal action: search, mutate, copy, compare, or produce a scalar/result object. Creating a view alone performs no traversal.

Range algorithms are constrained by concepts

Most of the useful checking happens at compile time.

```
template<ranges::random_access_range R,  
        class Comp = ranges::less,  
        class Proj = std::identity>  
requires std::sortable<ranges::iterator_t<R>, Comp, Proj>  
constexpr ranges::borrowed_iterator_t<R>  
    sort(R&& r, Comp comp = {}, Proj proj = {});
```

`random_access_range` checks traversal capability. `sortable` checks whether the projected elements can be reordered using the comparator.

Range concepts build on iterator concepts

```
template<class T>
concept random_access_range =
    ranges::bidirectional_range<T> &&
    std::random_access_iterator<ranges::iterator_t<T>>;
```

Range concepts obtain the iterator type and apply the C++20 iterator concepts. They do not rely only on the legacy iterator category tags.

Projections

A projection extracts a key before the comparator sees each element.

```
struct Lad {  
    std::string name;  
    int age;  
};  
  
std::vector<Lad> theLads = {  
    {"Erik", 77}, {"Bob", 33}, {"Charlie", 53}  
};  
  
std::ranges::sort(  
    theLads, std::ranges::greater{}, &Lad::age);
```

The projection may be a member pointer, function, or lambda. Here the comparator receives two `int` age values rather than two complete `Lad` objects.

Algorithm result types preserve useful state

Ranges algorithms often return a result object instead of discarding final iterators or callable state.

```
std::vector<int> output(values.size());  
auto [input_end, output_end] =  
    std::ranges::copy(values, output.begin());  
  
auto [last, function] = std::ranges::for_each(values, Counter{});  
std::cout << function.count;
```

Types such as `in_out_result<I, 0>` and `in_fun_result<I, F>` are aggregate results, so structured bindings expose their members directly.

Materializing a lazy pipeline

C++20 can copy a view into an owning container:

```
auto selected = values | std::views::filter(is_even);  
  
std::vector<int> stored;  
std::ranges::copy(selected, std::back_inserter(stored));
```

C++23 adds `std::ranges::to`:

```
auto stored = selected | std::ranges::to<std::vector>();
```

Materialize when results must own their elements, outlive the source, or be traversed repeatedly without reevaluating the pipeline.

ex4.cpp

Rewrite the order-processing workflow from `ex1.cpp` as a lazy C++20 ranges pipeline.

- Filter the orders whose status is "Completed".
- Transform each matching Order into its totalPrice.
- Chain both adaptors with the pipe operator (`|`).
- Consume the resulting view with `std::accumulate`.
- Verify that no intermediate `std::vector` is created.

The goal is to practise composing and consuming views, and to replace eager copying into an intermediate container with lazy evaluation.

Custom view and adapte

Create a moving average view. Given a range of numbers A. Produce a view that represents range A as the moving average range.

```
int main() {  
    std::vector<double> data = {1, 2, 3, 4, 5, 6};  
    auto avg3 = data | moving_average(3);  
    return 0;  
}
```

Define a struct 'moving_average'

Concept used to constrain templated parameter

This is the adaptor, the view factory.

```
struct moving_average {  
    std::size_t window_  
    friend auto operator|(std::ranges::viewable_range auto&& r, moving_average const& self) {  
        return moving_average_view(std::forward<decltype(r)>(r), self.window_  
    }  
};
```

```

template <std::ranges::input_range V>
requires std::ranges::view<V> && std::is_arithmetic_v<std::ranges::range_value_t<V>>
class moving_average_view
    : public std::ranges::view_interface<moving_average_view<V>>
{
    V base_;
    std::size_t window_;
    using T = std::ranges::range_value_t<V>;

public:
    template <std::ranges::viewable_range R>
    moving_average_view(R&& r, std::size_t w)
        : base_(std::forward<R>(r)), window_(w) {}
    struct iterator {
        ...
    };
    iterator begin() {
        return iterator(std::ranges::begin(base_),
                        std::ranges::end(base_),
                        window_);
    }
    std::default_sentinel_t end() const noexcept {
        return {};
    }
};

```

```

struct iterator {
    using value_type = T;
    using difference_type = std::ptrdiff_t;
    using iterator_category = std::input_iterator_tag;
    std::ranges::iterator_t<V> it_;      // current source iterator
    std::ranges::sentinel_t<V> end_;    // source range end
    std::deque<T> buf_;
    std::size_t window_;
    iterator() = default;
    iterator(std::ranges::iterator_t<V> it,
            std::ranges::sentinel_t<V> end,
            std::size_t window)
        : it_(it), end_(end), window_(window) {}
    value_type operator*() const {
        // produce a moving average from buf_
    }
    iterator& operator++() {
        ++it_;          // advance the source iterator
        // update buf_ with the latest value
        return *this;
    }
    friend bool operator==(const iterator& i, std::default_sentinel_t) {
        return i.it_ == i.end_; // the iterator ends when it reaches the source sentinel
    }
    friend bool operator==(std::default_sentinel_t, const iterator& i) {
        return i == std::default_sentinel_t{};
    }
}

```

The key idea is really simple:

- `it_` is the source iterator
- `end_` is the source sentinel
- the view's `end()` is the default sentinel of the view
- the relation `it_ == end_` decides when the custom iterator has exhausted the source range

Why the constructor takes a `viewable_range`

The constructor accepts a source range `R`, but the view stores `V`:

```
template<std::ranges::viewable_range R>
moving_average_view(R&& source, std::size_t window)
    : base_(std::views::all(std::forward<R>(source))),
      window_(window) {}
```

`viewable_range<R>` means `views::all` can safely turn the argument into a storable view:

- an existing view is copied or moved;
- a non-view lvalue becomes a `ref_view`;
- a movable non-view rvalue becomes an `owning_view`.

CTAD and std::views::all_t

CTAD rule, deduce parameter V from parameter R.

```
template <class R>
moving_average_view(R&&, std::size_t)
-> moving_average_view<std::views::all_t<R>>>;
```

std::views::all_t deduces to ref_view or owning_view depending on R.

```
int main() {
    std::vector<double> data = {1, 2, 3, 4, 5, 6};
    auto avg3 = data | moving_average(3);
    static_assert(std::same_as<decltype(avg3), moving_average_view<std::ranges::ref_view<std::vector<double>>>>>);
    auto avg4 = std::move(data) | moving_average(3);
    static_assert(std::same_as<decltype(avg4), moving_average_view<std::ranges::owning_view<std::vector<double>>>>>);
    return 0;
}
```

Adapter closure now only works in pipeline

Free pipeline operator --> inherit from `std::ranges::range_adaptor_closure`

```
struct moving_average : std::ranges::range_adaptor_closure<moving_average>
{
    std::size_t window_;
    auto operator()(std::ranges::viewable_range auto&& r) const {
        return moving_average_view(std::forward<decltype(r)>(r), window_);
    }
};

int main() {
    std::vector<double> data = {1,2,3,4,5,6};
    auto avg_pipe = data | moving_average{.window_ = 3};
    static_assert(std::same_as<decltype(avg_pipe), moving_average_view<std::ranges::ref_view<std::vector<double>>>>);
    moving_average ma{.window_ = 3};           // closure object
    auto avg3 = ma(data);                       // call operator() directly
    static_assert(std::same_as<decltype(avg3), moving_average_view<std::ranges::ref_view<std::vector<double>>>>);
}
```

ex5.cpp

Lazily produce the running total of the values seen so far.

- Implement `running_total_view`: `{10, 13, 12, 20}` becomes `{10, 23, 35, 55}`.
- Store the source position and accumulated total in an input iterator.
- Stop at the default sentinel when the source is exhausted.
- Add the `std::views::all_t` deduction guide.
- Complete the adaptor closure so `range | running_total` works.
- Verify that lvalues use `ref_view` and rvalues use `owning_view`.

The goal is to transfer the custom view pattern from the moving-average example to a stateful transformation that has no direct standard view equivalent.

Tombstone

C++ Ranges have a "Type-Safe Tombstone"

```
auto get_data() { return std::vector{1, 2, 3}; }  
int main()  
{  
    // std::ranges::find returns std::ranges::dangling.  
    auto it = std::ranges::find(get_data(), 2);  
    return *it;  
}
```

Runtime UB error intercepted by the compiler:

```
<source>: In function 'int main()':  
<source>:9:12: error: no match for 'operator*' (operand type is 'std::ranges::dangling')  
    9 |         return *it;  
      |                ^~~  
Compiler returned: 1
```

How the tombstone is selected

The range overload chooses its return type from the lifetime of the input.

```
struct dangling {  
    constexpr dangling() noexcept = default;  
  
    template<class... Args>  
    constexpr dangling(Args&&...) noexcept {}  
};  
template<std::ranges::range R>  
using borrowed_iterator_t = std::conditional_t<  
    std::ranges::borrowed_range<R>, /*iterators outlive containers */  
    std::ranges::iterator_t<R>,  
    dangling>;
```

dangling can be constructed from the iterator result, but deliberately has no dereference or increment operations.

A simplified range algorithm

The iterator overload does the work.

```
template<std::ranges::input_range R, class T>
borrowed_iterator_t<R> my_find(R&& range, const T& value) {
    auto result = std::ranges::find(
        std::ranges::begin(range),
        std::ranges::end(range),
        value);

    return result; // iterator, or converted to dangling
}
```

For an rvalue, R is not a borrowed range, so the return type is dangling. For span or string_view, iterators remain valid and are returned.

Why does `std::find` return dangling while `views::all` returns an `owning_view`?

- Algorithms return iterators. They cannot own the container.
- Views are objects. When given an Rvalue, they move the data into an `owning_view`.

```
//moving_average_view<std::ranges::owning_view<std::vector<int>>>>  
auto v = get_data() | std::views::filter(is_even);  
// 'it' cannot hold the vector. To prevent a crash, it becomes 'dangling'.  
auto it = std::ranges::find(get_data(), 2);
```

Borrowed ranges

Observation: Not all r-value ranges are unsafe

If the range dies, the iterators remain valid

```
auto it = std::ranges::find(std::string_view{"Hello"}, 'e');  
static_assert(!std::same_as<decltype(it), std::ranges::dangling>);
```

Propagating borrowing through a custom view

A wrapper view may be borrowed when its iterators remain valid after the wrapper object is destroyed. Often that follows the underlying view:

```
template<class V>
inline constexpr bool std::ranges::enable_borrowed_range<my_view<V>> =
    std::ranges::enable_borrowed_range<V>;
```

This is correct only when an iterator stores everything it needs, such as source iterators and copied state. Do not enable borrowing when iterators point back into the custom view for a predicate, cache, or other state.

The specialization changes algorithm return types; it does not extend the lifetime of underlying elements.

```
template<typename T>
struct MyCursor {
    T* ptr;
    T* begin() const { return ptr; }
    T* end() const { return ptr + 10; }
};
template<typename T>
inline constexpr bool std::ranges::enable_borrowed_range<MyCursor<T>> = true;
int main() {
    auto it = std::ranges::find(MyCursor<int>{nullptr}, 42);
    return *it;
}
```

```
template<std::ranges::input_range R>
decltype(auto) get_first(R&& r) {
    if constexpr (std::ranges::borrowed_range<R>) {
        return *std::ranges::begin(r); // Returns Reference (Fast)
    } else {
        using V = std::ranges::range_value_t<R>;
        return static_cast<V>(*std::ranges::begin(r)); // Returns Copy (Safe)
    }
}

int main() {
    decltype(auto) val = get_first(std::vector{1, 2});
}
```

Downside: views make lifetimes less obvious

This function returns a view containing a ref_view of a dead vector.

```
auto even_numbers_backwards() {  
    std::vector<int> numbers{1, 2, 3, 4, 5, 6};  
  
    return numbers  
        | std::views::filter([](int n) { return n % 2 == 0; })  
        | std::views::reverse;  
} // numbers is destroyed; the returned view now dangles  
  
for (int n : even_numbers_backwards()) {  
    std::cout << n << ' '; // undefined behaviour: possibly out of bounds  
}
```

The pipeline object survives. Its underlying data does not.

Similar syntax, different ownership

An rvalue container is moved into an owning_view; a named container is represented by a ref_view.

```
// Safe: the pipeline owns the temporary vector.  
auto safe = std::vector{1, 2, 3, 4}  
    | std::views::filter(is_even)  
    | std::views::reverse;  
  
// Dangerous when returned beyond numbers' lifetime.  
auto borrowed = numbers  
    | std::views::filter(is_even)  
    | std::views::reverse;
```

Do not infer ownership from the final view type being movable or returned as an rvalue. Ask what views::all did with the original source.

Cache correctness is a library concern

Forward-range views may cache positions to meet complexity guarantees. Those cached iterators must not accidentally survive a copy or move into different storage.

```
auto filtered = std::vector{1, 2, 3, 4} | std::views::filter(is_even);  
(void)filtered.begin(); // may populate a cache (iterator pointing to an element)  
auto reversed = std::move(filtered) | std::views::reverse; //invalidates cache
```

A conforming implementation uses a non-propagating cache or an equivalent safe representation (moving average view)

When does `filter_view` execute?

`begin()` and `end()` must be amortized constant time.

A filter cannot know its first element without testing the predicate.

For a forward range such as `vector`, `filter_view::begin()` therefore:

- scans only until the first match on its first call;
- caches that iterator for later calls to `begin()` ;
- evaluates the remaining elements as the filter iterator is incremented.

The filter is still lazy: calling `begin()` does not process the whole range.

```

#include <cassert>
#include <iostream>
#include <ranges>
#include <vector>

int main() {
    std::vector<int> data{1, 3, 5, 8, 10};
    int predicate_calls = 0;

    auto filtered = data | std::views::filter([&](int value) {
        ++predicate_calls;
        return value % 2 == 0;
    });

    std::cout << "after construction: "
              << predicate_calls << '\n';

    auto first = filtered.begin();
    std::cout << "after first begin: "
              << predicate_calls << '\n';
    std::cout << "first value: " << *first << '\n';

    auto again = filtered.begin();
    std::cout << "after second begin: "
              << predicate_calls << '\n';
    std::cout << "same position: "
              << std::boolalpha << (first == again) << '\n';

    ++first;
    std::cout << "after increment: "
              << predicate_calls << '\n';
    std::cout << "next value: " << *first << '\n';

    assert(predicate_calls == 5);
}

```

The output:

```
after construction: 0  
after first begin: 4  
first value: 8  
after second begin: 4  
same position: true  
after increment: 5  
next value: 10
```

Compile-time

- Heavy lifting is done by the type system.
- Building the pipeline is done at compile time
- Runtime; iterating over the items.
- You could get many specializations for a view (moving_average_view)
- Code size can explode
- Compile times can explode, any_view: P3411R0

A reproducible pipeline comparison

Both functions perform the same filter, transform, and reduction:

```
// Keep even values >= minimum, transform x -> 3*x + 7, then sum.
for (int value : values) {
    if (value >= minimum && value % 2 == 0)
        sum += transform_value(value);
}
```

```
auto transformed = values
| std::views::filter([minimum](int value) {
    return value >= minimum && value % 2 == 0;
})
| std::views::transform(transform_value);

for (std::uint64_t value : transformed)
    sum += value;
```

Benchmark controls

- Google Benchmark 1.9.4, GCC 15.2, C++23, `-O3`
- Allocation and input generation occur outside the timed loop
- Results are checked for equality before timing
- Both pipeline functions are `noinline`; the transform remains inlineable
- Nine repetitions of at least 0.2 seconds, medians reported
- Repetitions are randomly interleaved and pinned to one CPU
- Sizes: 1,024, 65,536, and 4,194,304 integers

Four patterns exercise different branch behavior:
all-pass, none-pass, alternating, and random-50.

One measured result

Pinned WSL/Docker run, 65,536 integers (256 KiB), median CPU time:

Input pattern	Raw loop	Ranges	Ranges / raw
all-pass	14.3 us	22.0 us	1.54x
none-pass	9.43 us	9.67 us	1.02x
alternating	11.9 us	17.9 us	1.50x
random-50	181 us	178 us	0.98x

Why do some results converge?

- none-pass: both implementations only test and reject until reaching the end;
`filter_view` never enters its match-consumption path;

- random-50: unpredictable branches dominate both implementations, masking

What survived optimization?

For this workload, GCC 15.2 produced the same output with normal `-O3` and with loop vectorization disabled: neither implementation was SIMD-vectorized.

The compiler did remove:

- adaptor objects;
- iterator wrappers;
- predicate and transform function calls.

The remaining loops were not identical. The raw version has one loop with an accept/reject path. The `filter_view` version alternates between searching for the next match and consuming a match. That extra control flow explains why the predictable accepting cases differ despite complete inlining.

Reproduce, then interpret

The complete experiment is in `benchmarks/ranges` :

- `pipeline.cpp` : the two functions used for timing and assembly
- `benchmark.cpp` : input patterns and Google Benchmark registration
- `pipeline_test.cpp` : semantic equivalence checks
- `generate_assembly.sh` : GCC and Clang assembly, with and without vectorization
- `README.md` : Docker-only configure, build, test, benchmark, and assembly steps

Zero-overhead does not promise identical assembly or identical timing. It means the abstraction can disappear; always measure the resulting program with representative data when the code is performance-critical.

C++23: zip parallel ranges

`std::views::zip` combines corresponding elements without creating a container. It is useful when data is stored as a structure of arrays.

```
std::vector<std::string> names{"Ada", "Bjarne", "Grace"};
std::vector<int> scores{91, 88, 95};

for (auto&& [name, score] : std::views::zip(names, scores)) {
    std::cout << name << ": " << score << '\n';
}
```

Each element is tuple-like and refers to the underlying elements. Mutating `name` or `score` here mutates the source range.

Zip stops at the shortest range

The end of a zip is the first end reached by any input range.

```
std::vector ids{10, 20, 30, 40};  
std::vector labels{"ten", "twenty"};  
  
auto rows = std::views::zip(ids, labels);  
assert(std::ranges::distance(rows) == 2);
```

This prevents out-of-bounds access, but a size mismatch may still be a domain error. Validate equal sizes separately when truncation would hide bad input.

Enumerate is a specialized zip

`std::views::enumerate` (C++23) pairs each element with a zero-based index.

```
for (auto&& [index, player] : std::views::enumerate(players)) {  
    std::cout << "Rank #" << index + 1 << ": " << player.name << '\n';  
}
```

Conceptually it resembles:

```
std::views::zip(std::views::iota(std::size_t{0}), players)
```

Use `enumerate` for positions; use `zip` when combining independent ranges.

More C++23 views

C++23 fills several common composition gaps:

View	Purpose
<code>views::slide(n)</code>	overlapping windows of exactly <code>n</code> elements
<code>views::chunk(n)</code>	consecutive non-overlapping blocks
<code>views::stride(n)</code>	every <code>n</code> th element
<code>views::chunk_by(pred)</code>	split when adjacent elements stop matching
<code>views::adjacent<N></code>	tuples of <code>N</code> adjacent elements
<code>views::adjacent_transform<N>(fn)</code>	transform adjacent tuples
<code>views::join_with(delimiter)</code>	flatten nested ranges with separators

Standard views replace common custom loops

```
#include <ranges>
#include <vector>
#include <iostream>
#include <algorithm>
int main()
{
    std::vector values = {10, 20, 30, 40, 50, 60};

    auto moving_average = values | std::views::adjacent_transform<3>([](int a, int b, int c) { return a + b + c; });
    std::ranges::for_each(moving_average, [](auto const& v){
        std::cout << v << std::endl;
    });
}
```

Replace a generic sliding-window view and a pairwise-difference view.

ex6.cpp

Revisit the channel image from `ex2.cpp` using C++23 views.

- Split the flat image into 4-byte channel chunks with `views::chunk`.
- Create lazy red, green, and blue streams using `drop` and `stride(3)`.
- Zip those streams so each iteration yields the three channel chunks from one image block.
- Use a structured binding to access the R, G, and B chunks.
- Verify the number of image blocks and their channel values.

The goal is to replace the custom channel iterator from `ex2.cpp` with composable views over the same interleaved storage.

Downside: wide zip types are expensive

`zip` is variadic. Every additional input becomes part of the view, iterator, sentinel, reference, value type, and constraints.

```
auto rows = std::views::zip(ids, names, ages, scores,  
                           teams, active, country, last_login);  
  
auto selected = rows  
    | std::views::filter([](auto&& row) { /* tuple-like proxy */ })  
    | std::views::transform([](auto&& row) { /* another view type */ });
```

The compiler instantiates operations across all component ranges. Generic lambdas and later adaptors add another layer of concepts and tuple machinery. Runtime traversal can still be cheap; compilation and diagnostics are the cost.

Why each zipped range adds work

For `zip_view<Views...>`, the library must compute and validate:

- the weakest iterator category supported by every input;
- whether all inputs are common, sized, random-access, or borrowed ranges;
- an iterator holding one iterator per input;
- a tuple-like proxy reference from every dereference;
- an end condition that stops when **any** input reaches its sentinel.

Keep wide pipelines behind an `auto`-returning function, avoid repeating their types in interfaces, and split a very wide zip when compile time becomes a measured problem.

Does zip compile time scale linearly?

There is no standard compile-time complexity guarantee. A flat zip of `N` ranges requires at least `N`-dependent work, but a pipeline can inspect that entire pack repeatedly.

```
auto rows = std::views::zip(r1, r2, /* ... */, rN); // N component ranges

auto result = rows
    | check_1    // constrains iterator/reference properties for all N inputs
    | check_2    // does so again
    // ...
    | check_N;   // N layers, each examining N components
```

That shape can cause roughly:

$$N + N + \dots + N = O(N^2)$$

Runtime and compile time diffe

For a zip of N ranges:

Operation	Typical runtime work
increment or dereference	$O(N)$
compare with end	$O(N)$
process M rows	$O(MN)$

"zero-copy" and "zero-overhead at runtime" do **not** mean cheap compilation.

Measure representative translation units with `-ftime-report` or Clang `-ftime-trace`; do not assume compile time is $O(N)$.

ex7.cpp

Implement `budget_batches_view`, which greedily groups adjacent costs without exceeding a budget.

- Require a const `forward_range` and `common_range` source.
- Produce `{4, 2}`, `{5}`, `{3, 1}` from `{4, 2, 5, 3, 1}` with budget 7.
- Track each batch's beginning and end while guaranteeing forward progress.
- Expose each batch as a `std::ranges::subrange` without allocating.
- Propagate `borrowed_range` only when the underlying view is borrowed.
- Verify both safe iterator returns and `std::ranges::dangling`.

The goal is to implement domain-specific lazy grouping while practising range constraints, subrange-valued iteration, and conditional borrowing.

ex8.cpp

Build a ranked leaderboard using a range action followed by a lazy pipeline.

- Sort players by descending score using `std::ranges::sort`.
- Use `std::ranges::greater` and the `&Player::score` projection.
- Filter out players scoring below 1000.
- Enumerate the remaining players to generate zero-based positions.
- Transform each tuple-like result into the expected rank string.
- Verify the resulting order, ranks, and values.

The goal is to combine eager algorithms with `filter`, `enumerate`, and `transform`, while using projections and tuple-like proxy references correctly.

ex9.cpp

Implement a `trim_view` that lazily removes matching elements from both ends.

- Require a bidirectional, common source range and a valid predicate.
- Find the first non-matching element with `std::ranges::find_if_not`.
- Walk backward to find the end without decrementing before `begin()`.
- Add a `std::views::all_t` deduction guide.
- Implement direct-call and pipe syntax through an adaptor closure.
- Verify normal, empty, and all-matching ranges.

The goal is to practise a bidirectional custom view, safe boundary handling, and a reusable range adaptor closure.

We bring **high-tech** to life